Configuration Files

Using settings for configuring parts of an Unreal Engine application.

Configuration Files or **Config Files** provide the settings for **Unreal Engine (UE)**. An User must keep track, thus, control of the rules of the value-pairs organized into sections. These files are used to an Unreal Engine application, or a game, to read settings and preferences at runtime. Read more about Configuration Files and the `Config` Specifier.

Syntax

Configuration files are structured as follows:

```
[SECTION]
[SECTION:SUBSECTION]
[SECTION:SUBSECTION:SUBSUBSECTION]
key=value
key=value
key=value
```

Copy Snippet

Key-Value pairs, which are separated by a single equal sign, or `key=value`.

Section Names

Section names are alphanumeric strings you can set to any value. The Config system treats all sections in a configuration file as a single, unordered, collection of key-value pairs. For example, if you write a category named `ExampleSection`, write:

```
[ExampleSection]
```

Copy Snippet

Modules

Section headings for configuration objects contained in modules use the syntax:

```
[/Script/ModuleName.ClassName]
```

Copy Snippet

- `ModuleName`: the name of the module containing the configuration object.
- `ClassName`: the name of the class within the module `ModuleName`, containing the configuration object.

Plugins

Section headings for configuration objects contained in plugins use the syntax:

```
[/PluginName/ClassName]
```

Copy Snippet

- `PluginName`: the name of the plugin containing the configuration object.
- `ClassName`: the name of the class within the plugin `PluginName`, containing the configuration object.

Blueprints

Section headings for configuration assets contained in blueprints use the syntax:

```
[/GameFolder/AssetNameBP.AssetNameBP_C]
```

Copy Snippet

- `GameFolder`: the path to your asset.
- `AssetNameBP`: the name of your asset, `AssetNameBP_C` containing the configuration object.

Custom Section Names

Section names can also be used as a custom category name. For example, if you wrote a category named `ExampleSection`, write:

```
[ExampleSection]
```

Copy Snippet

When you use custom section names, you must manually query the config variables contained in the section. See the Manual Access Configuration Settings in Scripts section for more information.

Comments

Anything after a `;` (semicolon) is treated as a comment in your configuration file. This also works for commenting out key-value pairs as opposed to deleting them from a file.

```
; This section
; This section can be used to Defaultengine.ini to override the display level for AssetTool
[AssetTool]
; Display level
DisplayLevel=Verbose
```

Copy Snippet

Key-Value Pairs

Configurations can represent a variety of different types of information.

- Strings
- Numbers
- Arrays
- Structs

Arrays

Arrays in Unreal Engine configuration files provide a mechanism for you to combine, accumulate, or override information across multiple configuration files in the configuration hierarchy. For example, you can add multiple entries to a single configuration array object across multiple files in the configuration hierarchy, such as `Engine.ini`, `DefaultEngine.ini`, or your engine directory, and `MyProjectEngine.ini` in your project directory.

Configuration files support the following array operations:

Operation	Operator	Example
Empty	`!`	Empties the contents of an array, then adds a value. `!` is optional. For clients, the recommended syntax is `+array=()`, or `+array=(%s,%s,%s)`. `+array=()`, `+array=(%s,%s,%s)`
Append	`+`	Appends the value to the end of the array. If does not yet exist, it is also added. `+` is optional. `+array=(%s)` `array=(%s)`
Remove	`-`	Removes the value from the array. Must be an exact match. `-array=(%s)`
Append If Unique	`.`	Appends the value to the array, unless it is already present in the array. `.array=(%s)`

Array Example

```
MyArrayExample=1
MyArrayExample=2
MyArrayExample=3
MyArrayExample=4
MyArrayExample=5
MyArrayExample=6
MyArrayExample=7
```

Copy Snippet

The files in this example perform the following operations:

- `MyArrayExample=1` creates the array for the first element 1.
- `!MyArrayExample=(MyExample)` clear the array of all entries.
- `+MyArrayExample=2` append 2 to the array.
- `+MyArrayExample=3` append 3 to the array.
- `+MyArrayExample=4` append 4 to the array.
- `-MyArrayExample=3` remove element number 3 in the array.
- `+MyArrayExample=5` remove 5 from the array.

In the end, `MyArrayExample` contains `(1,2,4)`.

Structs

Configuration files support changing struct properties in a config object. The syntax for setting a struct is:

```
[/Script/MyModule.MyClass]
MyStructMyMemberMyStructMyMemberMyStructMyMemberM
```

```
-<CATEGORY>Ini=<FILE_NAME>
```

Copy full snippet

Description

Override all configuration files in the given `CATEGORY`. For a full list of available configuration file categories, see the Configuration File Categories section of this page.

Example

```
-EngineIni=FooEngine.ini
```

Copy full snippet

`FooEngine.ini` overrides all other `*Engine.ini` files. For a full list of available configuration file categories, see the Configuration File Categories section of this page.

Config File with Matching Suffix

Command

```
-iniFile=<PATH/TO/FILE_NAME>
```

Copy full snippet

Description

Override the config file with the matching suffix path.

> This command requires that your path's suffix directory structure match the engine directory structure for configuration files.

Example

```
-iniFile=C:/MyAdditionalConfigFiles/Engine/Config/BaseEngine.ini
```

Copy full snippet

`C:/MyAdditionalConfigFiles/Engine/Config/BaseEngine.ini` overrides the configuration file `<PROJECT_DIRECTORY>/Engine/Config/BaseEngine.ini`. For a full list of available configuration file categories, see the Configuration File Categories section of this page.

All Config Files

Command

```
-IniBootstrap=<FILE_NAME>
```

Copy full snippet

Description

Override all config files being loaded and parsed. Read-only.

Example

```
-IniBootstrap=Foo.ini
```

Copy full snippet

`Foo.ini` is the only configuration file loaded, nothing else.

Set Console Variables in Configuration Files

Configuration Files provide another mechanism for specifying console variables. Some console variables can be set in general configuration categories whereas others should be set in specific configuration categories. Generally, console variables are set in the `[ConsoleVariables]` section in `DefaultEngine.ini` within your project directory.

Specific Categories

The following table describes the specific categories of console variables that should be set in particular sections:

Type	Section	Description
Rendering	`[/Script/Engine.RendererSettings]`	Any console variable starting with `r.`
Rendering Override	`[/Script/Engine.RendererOverrideSettings]`	Specifically for the console variable `r.SupportAllShaderPermutations`
Streaming	`[/Script/Engine.StreamingSettings]`	Any console variable starting with `s.`
Garbage Collection	`[/Script/Engine.GarbageCollectionSettings]`	Any console variable starting with `gc.`
Network Settings	`[/Script/Engine.NetworkSettings]`	Only for the console variables `n.VerifyPeer`, `p.EnableMultiplayerWorldOriginRebasing`, and `NetworkEmulationProfiles`.
Cooker Settings	`[/Script/UnrealEd.CookerSettings]`	Any console variable starting with `cook.`

Useful Source Files for More Information

The following engine files provide more information about the config system and its components:

- ConfigCacheIni
- CoreGlobals
- ConfigHierarchy

configuration programming config

Ask questions and help your peers Developer Forums

Write your own tutorials or read those from others Learning Library

Collapse